VIETNAM NATIONAL UNIVERSITY, HANOI

INTERNATIONAL SCHOOL

REPORT OF FINAL EXAM PROJECT

Topic: Patient Waiting List Management System

Subject: **Data Structures and Algorithms**

Class: **INS305002**

Lecture: **Trần Thị Ngân**

Group 17

Name	**Student ID**
Lê Hoàng Anh	23070692
Bùi Tố Uyên	23071109
Chimaobi Chukwuemeka Prince	22070004
Đỗ Thị Phương	23070615
Nghê Thị Thu Thảo	24070902

Hanoi, June 2026

CONTRIBUTION TABLE

Name	Student ID	Task	Contribution
Lê Hoàng Anh	23070692	I. Introduction + V. Conclusion III.1. Patient Management Module - (Add New Patient + Search + Update)	**20%**
Bùi Tố Uyên	23071109	II.1. Problems analysis III.1. Patient Management Module (Delete)	**20%**
Chimaobi Chukwuemeka Prince	22070004	III.3. Queue Processing Module	**20%**
Đỗ Thị Phương	23070615	II.2. System Design IV. Result	**20%**
Nghê Thị Thu Thảo	24070902	III.2. Appointment Management Module	**20%**

TABLE OF CONTENTS

LIST OF FIGURE

I. Introduction

1. State the problem

In modern healthcare environments, managing patient information and scheduling is an essential task that directly affects service quality and operational efficiency. Currently, many clinics still rely on fragmented paper records or disparate Excel files. This manual approach leads to significant operational challenges, including difficulty in retrieving medical histories, a high risk of data loss, and severe scheduling conflicts where multiple patients are inadvertently booked for the same doctor at the exact same time [1]. Furthermore, traditional methods often fail to dynamically prioritize vulnerable demographic groups, such as the elderly or pregnant women, resulting in inefficient waiting room management. Recognizing that healthcare is a critical service, our team chose this topic to digitize and streamline the clinical workflow. By applying information technology, we aim to reduce the administrative burden on receptionists, eliminate scheduling overlaps, and ensure a seamless, equitable experience for patients [2][3][4].

From the perspective of data structures and algorithms, patient records constitute a dynamic dataset that requires frequent insertion, deletion, updating, searching, and reordering. The choice of data structure therefore has a direct impact on system performance and maintainability. Static structures such as arrays may be less appropriate in this context because they require fixed allocation and can incur additional overhead when records are inserted or removed. Consequently, this project addresses the problem of designing a patient waiting list management system capable of handling dynamic patient data efficiently while supporting appointment scheduling, priority-based queue management, and persistent data storage.

2. Objectives

This project aims to develop a Patient Waiting List Management System using C++ as a practical application of fundamental data structures and algorithms. The system provides essential functionalities for managing patient records, including registration, information retrieval, updating, deletion, appointment scheduling, doctor availability checking, waiting list visualization, and data persistence.

A key objective is to demonstrate how appropriate data structures can support the management of dynamic healthcare data. The implementation employs a singly linked list to maintain patient records and STL vectors to store appointment information associated with each

patient. In addition, the system incorporates file handling techniques for persistent storage and applies Merge Sort to organize the waiting list according to predefined priority criteria. Through these design choices, the project illustrates the relationship between algorithmic concepts and real-world information management problems.

3. Tools and methods

The Patient Management System was developed using C++ as the primary programming language, utilizing the Standard Template Library (STL) such as <vector> for managing dynamic appointment lists, <string> for input processing, and <fstream> for data streaming [5][6]. To ensure cross-platform compatibility and stability, the development and testing phases were conducted across multiple operating systems.

The team uniformly utilized Visual Studio Code (VS Code) as the primary code editor. For compilation and execution, macOS environments utilized Clang/LLVM via automated shell scripts directly in the Terminal, while Windows environments relied on the MinGW (GCC) compiler. This dual-platform approach guaranteed consistent input buffer handling and execution stability regardless of the hardware used.

In terms of system design, the project adopts a Procedural Programming paradigm combined with a modular architecture. Complex algorithms and business logic were broken down into sequential, manageable functions, distributed across logically separated source files. A central header file was established to manage core definitions and master data, while independent files handled specific operational modules such as patient registration, appointment scheduling, and queue sorting.

For data representation, the system utilizes struct to model real-world entities like patients and medical staff. Furthermore, the system implements a database simulation. All historical records are serialized and stored in text files (.txt), allowing the system to efficiently parse, store, and reconstruct data upon execution without the need for an external database engine.

II. Problem analysis and Designing

1. The problem of selecting a data structure

To effectively manage the dynamic and unpredictable nature of healthcare records, the system architecture requires a data structure that balances memory efficiency with operational

flexibility. After analyzing the computational context, the team opted for a Singly Linked List as the primary framework, integrated with a Hybrid Data Modeling approach.

A singly linked list is a linear data structure in which elements are stored as nodes connected through pointers. Each node consists of two parts: a data field and a link (pointer) that stores the address of the next node in the list (Debdutta Pal & Suman Halder, 2018). Unlike arrays, linked list nodes are not stored in contiguous memory locations. Instead, they are allocated dynamically in memory. The link between nodes enables the formation of a sequence, where each node points to its successor, and the last node contains a NULL pointer indicating the end of the structure.

Singly linked lists exhibit several crucial characteristics that perfectly align with the operational requirements of a clinical waiting list:

- Dynamic Size: In a clinic, the volume of incoming patients fluctuates unpredictably. The linked list can grow or shrink during runtime without requiring complex memory reallocation, easily accommodating sudden influxes of patients.
- Efficient Insertion and Deletion: Clinical workflows require frequent updates, such as inserting a high-priority patient (e.g., a pregnant woman) to the front of the queue or removing a discharged patient. These operations are performed by simply updating pointer references, avoiding the high computational cost of shifting elements required.
- Memory Utilization Flexibility: Nodes are allocated individually, allowing the structure to operate effectively even under fragmented memory conditions typical of systems running continuously over long shifts.

Despite its operational advantages, the singly linked list presents certain limitations, such as no random access when elements cannot be accessed directly by index, requiring sequential traversal from the head node and additional memory overhead (due to the storage of pointers). However, for the expected scale of this system, the traversal cost is negligible and entirely acceptable. It does not justify the implementation of more memory-intensive structures like Hash Tables.

To overcome the unidirectional limitation of the linked list and optimize data grouping, the system employs a hybrid structure [7]. While the core patient roster is managed via the Singly Linked List, where each Node contains a Patient struct, the appointment history associated with each patient is managed using the C++ STL std::vector. This creates a

One-to-Many relationship model within a single node. The vector allows for rapid, index-based access to a specific patient's historical appointments, perfectly balancing the dynamic nature of the patient queue with the structured, sequential nature of medical records.

2. System design

2.1. Libraries

In the development of the Patient Waiting List Management System using the C++ programming language, several standard libraries were utilized to support input/output operations, dynamic data storage, string processing, scheduling management, sorting operations, and conflict validation.

The <iostream> library is used to handle all console input and output operations within the system. Functions such as cout are used to display menus, waiting lists, doctor schedules, and patient information to users, while cin is used to capture keyboard input during system interaction.

The <vector> library is one of the core components of the system architecture. It provides a dynamic container for storing appointment records automatically without requiring manual memory allocation. The vector structure supports automatic resizing through operations such as push_back(), allowing appointment histories to expand dynamically whenever new bookings are added. This approach eliminates the need for manually managing memory using malloc(), realloc(), or free().

The <string> library is used for handling textual data such as patient names, doctor names, departments, appointment dates, and time slots. Unlike traditional character arrays in the C language, the string class provides safer and more flexible string manipulation while reducing buffer overflow risks and simplifying comparison operations.

The <algorithm> library is used to support sorting and scheduling operations. Functions such as sort() are utilized to organize waiting lists according to hospital priority rules, registration order, and appointment times.

The <fstream> library supports file input and output operations for persistent data storage. The system uses file streams to save and restore patient and appointment information between program executions.

The <ctime> library is used to handle timestamp generation and scheduling time management. Functions such as time(NULL) are utilized to generate registration timestamps for appointments, allowing the system to compare booking order efficiently during waiting list sorting operations.

The <iomanip> library is used to improve the formatting and readability of displayed appointment schedules, waiting lists, and patient records within the console interface.
Overall, these libraries provide the fundamental tools required to build a scalable, efficient, and maintainable hospital waiting list and appointment management system.

2.2. Patient and Appointment Data Structures

In the Patient Waiting List Management System, organizing scheduling information in a structured and logical manner is essential for managing appointments, doctor allocation, waiting queues, and patient prioritization efficiently. To achieve this, the system uses struct declarations in C++ to define custom data types representing appointments, patients, and waiting list nodes. The Appointment structure stores information related to a specific medical appointment. Each appointment contains the appointment date, appointment time, requested medical department, assigned doctor name, queue number, appointment status, and registration timestamp.

```
struct Appointment {
    string appointmentID;
    string date;
    string timeSlot;
    string department;
    string doctorName;
    int queueNumber;
    string status;
    time_t registrationTimestamp;
};
```

This structure represents a complete appointment record within the hospital scheduling system. The registrationTimestamp variable stores the exact booking time of an appointment using the time_t data type. This design simplifies waiting list sorting because the system can

directly compare timestamp values numerically when determining which patient registered earlier.

The structure allows the system to track appointment allocation, doctor scheduling, queue ordering, and waiting list status efficiently. The Patient structure stores the personal information of each patient. Each patient record includes the patient ID, full name, age, phone number, pregnancy status, and automatically assigned priority level.

```
struct Patient {
    string id;
    string name;
    int age;
    string phoneNumber;
    bool isPregnant;
    int priorityLevel;
    vector<Appointment> appointments;
};
```

The system automatically determines the priority level according to hospital scheduling policies:

+ Priority Level 1:
  - Patients under 5 years old
  - Patients above 70 years old
  - Pregnant patients.
+ Priority Level 2: Normal patients.

Instead of using manually managed dynamic arrays, the system utilizes the Standard Template Library (STL) vector container to store multiple appointments for each patient. The vector<Appointment> structure automatically manages memory allocation internally and dynamically expands whenever new appointments are inserted using the push_back() operation. This significantly simplifies memory management and improves scalability compared to traditional dynamic array implementations.

To globally manage all patient records dynamically, the system uses a singly linked list structure. Each patient is wrapped inside a Node structure containing patient data and a pointer to the next node.

This hybrid architecture combines the advantages of singly linked lists and STL vectors:

- The singly linked list efficiently manages the global patient waiting queue dynamically.
- The STL vector efficiently manages multiple appointment records for each patient.

Overall, the combination of structures, linked lists, and STL vectors provides a flexible, scalable, and memory-efficient architecture suitable for real-world hospital waiting list and appointment scheduling systems.

2.3. Node Structure and Vector-Based Appointment Management

To manage a dynamic collection of patient records, the system employs a singly linked list data structure. In this design, each element in the waiting list is represented as a node containing patient information and a pointer to the next node in the sequence.

Each node consists of two main components:

- a data field storing a Patient structure,
- and a pointer field named next storing the address of the next node in the linked list.

These nodes are connected sequentially through the next pointers, forming a chain-like structure for dynamically managing patient records and waiting queues.

The linked list operates through a head pointer named head, which points to the first node in the waiting list. If the list is empty, the head pointer is assigned the value NULL. To traverse the list, the system begins from the head node and continuously follows the next pointers until reaching a node whose next pointer is NULL, indicating the end of the list.

The use of a singly linked list is highly suitable for this system because the number of patients within the waiting queue is not fixed and may change continuously during program execution. Unlike static arrays, linked lists support dynamic insertion and deletion without requiring memory shifting operations, making them efficient for real-time waiting list management.

In addition to the linked list structure, each Patient object maintains an STL vector named appointments for storing multiple appointment records. The vector container automatically expands its internal memory whenever new appointments are added through push_back(). This allows the system to support long-term appointment history management without requiring manual memory resizing or capacity tracking.

This creates a hybrid architecture in which:

- The global patient waiting list is managed using a singly linked list.
- While each patient's appointment history is managed independently using STL vector containers.

This design is highly appropriate for hospital scheduling systems because appointment records frequently require: chronological sorting, appointment insertion, doctor schedule tracking, queue calculation, duplicate booking detection and scheduling conflict validation. The system also implements prioritization and waiting list sorting mechanisms. Patients are organized according to: priority level, registration timestamp and appointment time.

Priority patients are always processed before normal patients. Within the same priority group, patients who register earlier are assigned earlier queue positions.

The system additionally supports doctor scheduling management. When a patient selects a medical department such as Neurology, Internal Medicine, Surgery, or Gastroenterology, the system checks for available doctors within the corresponding department and prevents scheduling conflicts by ensuring that a doctor cannot be assigned to multiple patients during the same time slot. Furthermore, when a patient is dequeued or deleted from the system, the delete operator in C++ automatically invokes the destructor of the STL vector, safely freeing the dynamically allocated memory for all appointment records and preventing memory leaks.

However, singly linked lists also have certain limitations. Accessing a specific patient record requires sequential traversal from the beginning of the list, resulting in a time complexity of $O(n)$ for searching operations. Despite this drawback, the flexibility, scalability, and efficient memory utilization of linked lists make them highly suitable for this Patient Waiting List Management System.

2.4. Program Flow

The overall system is designed using a menu-driven approach, where users interact with the Patient Waiting List Management System through a console-based interface. The program flow follows a structured sequence consisting of initialization, data loading, appointment processing, waiting list management, scheduling validation, file storage, and system termination.

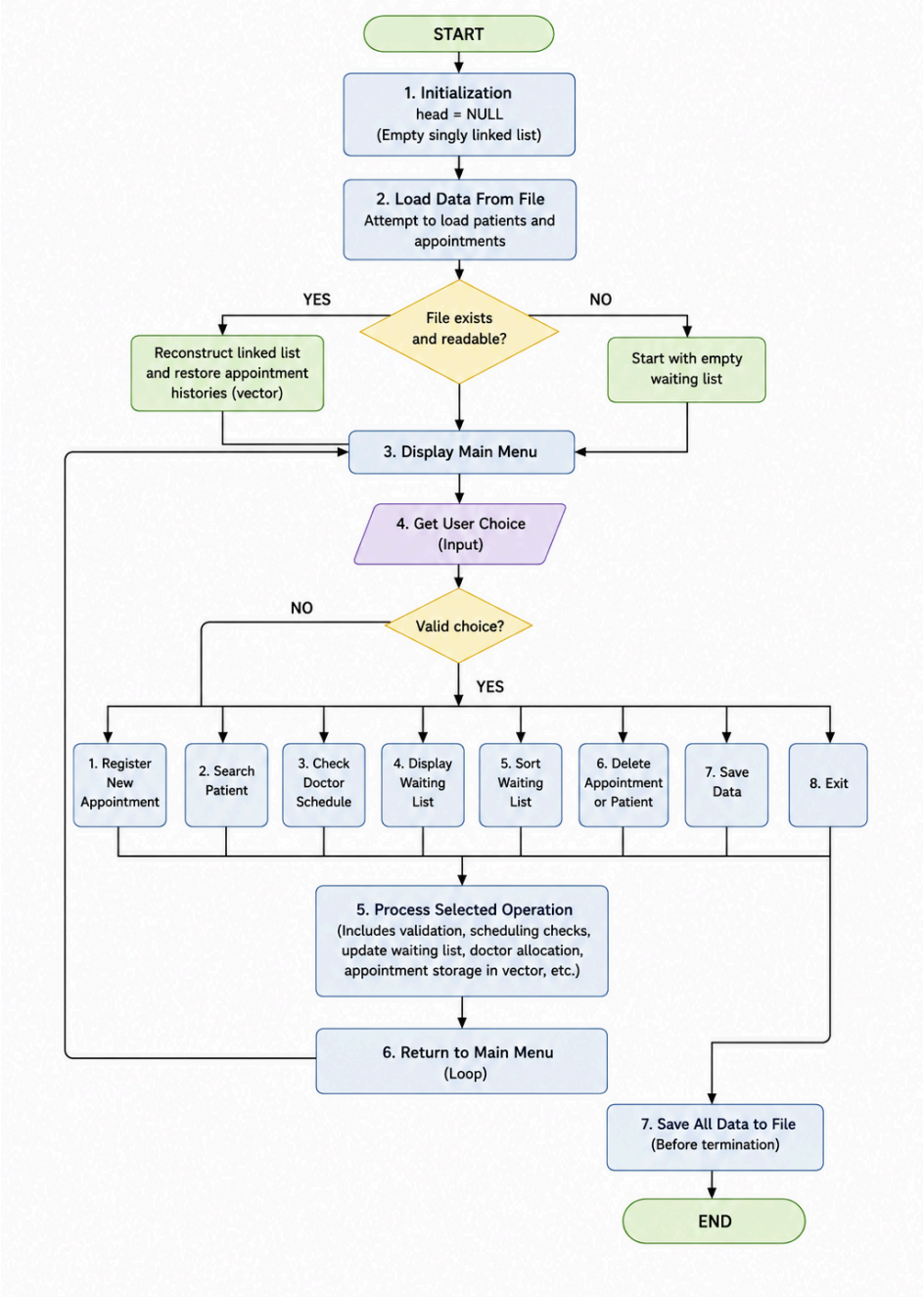

Figure 1: Program Flowchart

At the beginning of program execution, the head pointer of the singly linked list is initialized to NULL, indicating that the patient waiting list is initially empty. The system then attempts to load previously stored patient and appointment records from external storage files. If the files exist, the system reconstructs the linked list dynamically and restores all appointment histories into STL vector containers. If the files do not exist, the system starts with an empty

waiting list and continues execution normally. After initialization, the system displays the main menu containing several core operations, including:

- patient appointment registration,
- searching patient records,
- checking doctor schedules,
- displaying waiting lists,
- sorting appointments,
- deleting appointments,
- saving records,
- and exiting the system.

The user selects an operation through keyboard input. Before executing any operation, the system validates the input to ensure that the selected menu option is valid. Invalid inputs cause the system to return safely to the main menu without terminating the program.
When registering a new appointment, the system first collects patient information such as patient ID, patient name, age, phone number, pregnancy status, appointment date, appointment time, and the requested medical department.

Based on hospital scheduling policies, the system automatically determines whether the patient belongs to the priority group. Patients under 5 years old, patients above 70 years old, and pregnant patients are automatically assigned Priority Level 1, while all other patients are assigned Priority Level 2.

After collecting appointment information, the scheduling module performs several validation checks, including: doctor availability checking, duplicate appointment detection, overlapping appointment validation and queue conflict checking.

If a selected doctor already has an appointment during the same time slot, the system automatically rejects the booking request or searches for another available doctor within the same department.

Once validation is completed successfully, the appointment is inserted into the patient's STL vector appointment history using the push_back() operation, and the patient is placed into the waiting list according to the scheduling priority rules. This scheduling mechanism ensures that priority patients are processed before normal patients while still maintaining fairness among patients within the same category.

The system also supports doctor schedule tracking functionalities. Users can check a doctor's appointments for a specific day, determine a patient's queue number, calculate the number of patients waiting ahead and verify appointment schedules dynamically.

After each operation is completed, the program returns to the main menu and waits for the next user command. This interaction loop continues until the user selects the exit option. Before program termination, the system saves all patient and appointment information into external storage files to ensure persistent data management across multiple executions. Since the system utilizes STL vectors and C++ automatic object management, memory cleanup is handled automatically by the language runtime, significantly reducing the risk of memory leaks compared to manual memory management approaches.

Overall, this program flow provides a structured and scalable mechanism for managing hospital waiting lists, doctor allocation, appointment scheduling, queue prioritization, and scheduling conflict detection efficiently.

2.5. Function Prototypes and Pointer Convention

In addition to data structures and program flow, defining clear function prototypes and maintaining a consistent pointer-passing convention are essential for ensuring the correctness, scalability, and maintainability of the Patient Waiting List Management System.

Function prototypes specify the function name, return type, and parameter types before actual implementation. These declarations act as contracts within the program, allowing the compiler to perform type checking and ensuring consistent interaction between different modules.

The system adopts the following function prototypes. Each prototype represents a core operation related to linked list management, appointment scheduling, waiting list processing, conflict detection, doctor allocation, file handling, and data validation.

+ The createNode() function dynamically creates a new linked list node containing patient information.
+ The addPatient() function inserts a new patient node into the singly linked list. Since the head pointer may change during insertion, the function uses a double pointer (Node**).
+ The searchPatient() function traverses the linked list sequentially to locate a patient record based on patient ID. Since this operation only reads data, a single pointer (Node*) is sufficient.

+ The registerAppointment() and addAppointment() functions manage appointment creation and insertion into the patient's STL vector appointment history.

The scheduling validation functions are responsible for preventing overlapping doctor schedules, duplicate bookings, unavailable appointment slots and invalid scheduling conflicts. This ensures that high-priority patients are processed first while still maintaining fairness within the same scheduling category.

Overall, the system follows a consistent pointer convention throughout the implementation:

- Functions that modify linked list structures use double pointers (Node**),
- Functions that only traverse or display linked list data use single pointers (Node*),
- Appointment management operations use references (Patient&) for direct object modification,
- Validation functions operate directly on primitive values and STL string objects.

This convention improves readability, reduces pointer-related programming errors, and ensures safer memory management throughout the entire system.

III. Build the system

1. Patient Management Module

1.1. Add New Patient

The Add New Patient execution workflow proceeds through four distinct phases.

- Phase 1: Initialization & Auto-ID Generation

When the user selects Option 1, the system instantiates a temporary *Patient structure (p)*. Instead of requesting manual ID input, the system invokes the *generateNextPatientID(head)* function. This algorithm traverses the existing linked list, parses the current IDs from string to integer, identifies the maximum ID value, and increments it by 1. This guarantees a sequential and universally unique identifier without user intervention.

- Phase 2: Data Input & Strict Validation

The system employs *while(true)* loops to create "validation gates" for user inputs:

+ Phone Number: The input is stripped of trailing spaces using *trimInput()*. It is then passed to *isValidPhone()* to ensure it contains exactly 10-11 digits and starts with a '0'.

Furthermore, *searchPatientByPhone()* is called to verify that the phone number is not already registered in the system, effectively preventing duplicate patient records.

+ Name: The system validates that the string is not empty
+ Age: The input is parsed and validated to ensure it falls within a logical human lifespan (1 < age < 150)
+ Pregnancy Status: A simple boolean flag (true/false) is recorded based on user input.

- Phase 3: Priority Assessment

Before saving, the system passes the validated *age* and *isPregnant* status to the *determinePriorityLevel()* function. According to clinical rules, if the patient is a child (age < 5), elderly (age > 70), or pregnant, the function returns a Priority Level of 1 (High). Otherwise, it defaults to 2 (Normal). This value is permanently saved to the patient's record to facilitate future queue sorting.

- Phase 4: Dynamic Memory Allocation & Linked List Insertion

Finally, the system calls *addPatient(&head, p)*. Inside this function, *createNode(p)* is executed. This utilizes the new keyword to dynamically allocate memory in the RAM for a new Node, storing the patient data and setting the next pointer to nullptr.

The system then traverses the Singly Linked List from the head to find the last node. Once found, it updates the tail's next pointer to link to the newly created node in O(n) time complexity. The new patient is now officially registered in the system.

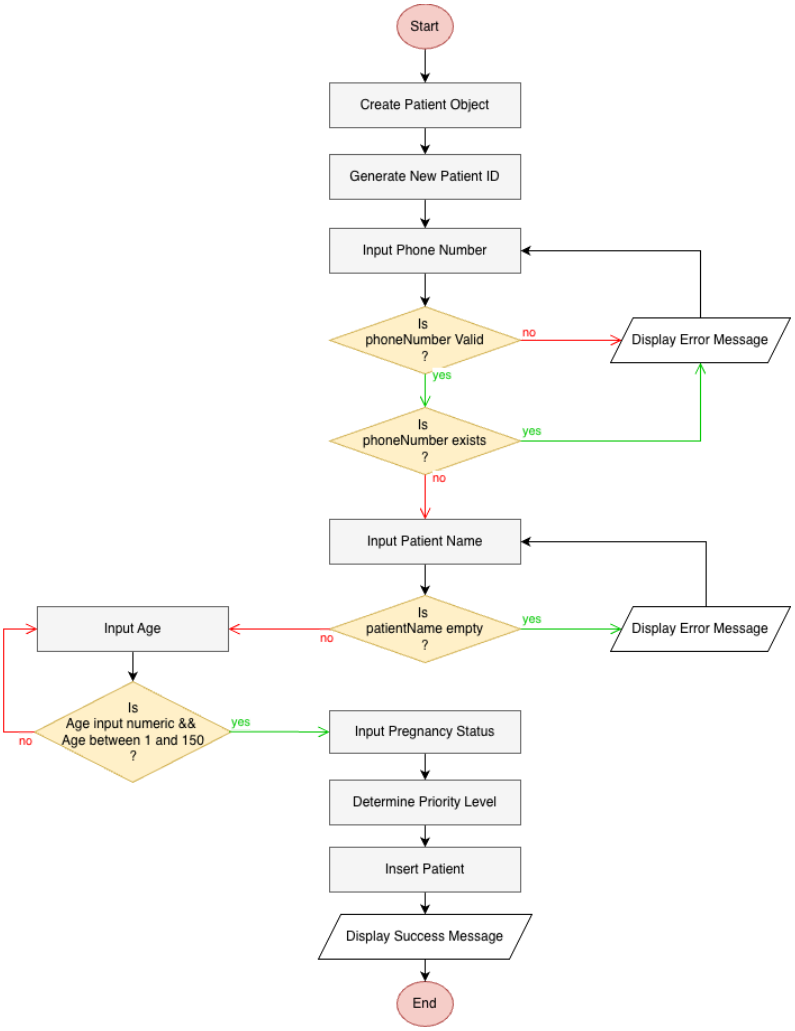

Figure 2: Add New Patient Flowchart

1.2. Search Patient

The "Search Patient" workflow is engineered to retrieve specific patient records efficiently, where patients are most easily identified by their phone numbers. The core mechanism relies on the Linear Search algorithm applied to a Singly Linked List, operating with a worst-case time complexity of O(n). The execution flow is structured into three main phases:

- Phase 1: Input Processing and Sanitization

The system prompts for a target phone number. To ensure search accuracy and prevent mismatch errors caused by accidental keystrokes, the system first captures the input using getline() and passes it through the trimInput() helper function. This strips any leading or trailing whitespaces, ensuring a clean string is prepared for the matching process.

- Phase 2: List Traversal and String Matching (Linear Search)

The clear string is passed as an argument to the searchPatientByPhone(Node* head, string phoneNumber) function. The algorithm initializes a temporary pointer (Node* temp) and points it to the head of the linked list. A while loop is deployed with the condition temp != nullptr, allowing the pointer to traverse the list sequentially from the first node to the last.

At each iteration, the system performs a string comparison between the target phone number and the current node's data (temp->data.phoneNumber == phoneNumber): If a match is found, the function immediately terminates the loop and returns the memory address (pointer) of that specific node. If the current node is not a match, the pointer advances to the next node using the assignment temp = temp->next.

- Phase 3: Result Evaluation and Display

Once the search function completes, the main menu evaluates the returned pointer.

+ If the pointer is nullptr, the loop reaches the end of the list without finding a match, the system outputs a localized error message informing that the patient does not exist in the database

+ If a valid pointer is returned, the system successfully accesses the memory block and extracts the patient's details. It then displays a comprehensive overview of the patient's demographic information (Name, Age, Priority Level, Pregnancy Status) alongside a summary of their booked appointments, allowing the receptionist to proceed with further clinical workflows.

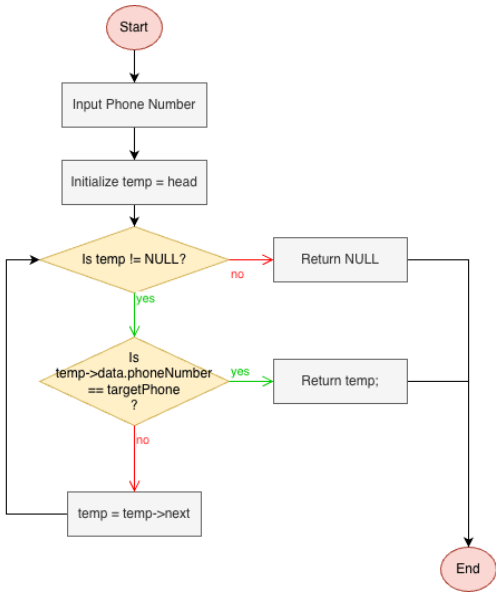

Figure 3: Search Patient by Phone Number Flowchart

1.3. Update Patient Information

The "Update Patient" workflow is designed with strict adherence to data integrity and system stability principles. It allows administrative staff to modify demographic details while safeguarding the core architectural constraints of the database. The execution flow is divided into four phases:

- Phase 1: Record Identification and Key Immutability

The process initiates by prompting the user for the Patient ID. To prevent the notorious buffer overflow issues, the system strictly utilizes getline() for ID capture. The system then calls the searchPatient() function to locate the corresponding node. A crucial design decision here is the Patient ID serves as the primary key and is strictly read-only. Modifying the ID is programmatically blocked to prevent cascading failures in the linked appointment records.

- Phase 2: State Presentation and Smart Input Handling

Once the patient node is located, the system displays the current demographic state to provide a baseline for the user. The system implements a "Press Enter to Keep" mechanism. Temporary variables are initialized with the patient's existing data. When the system prompts for new inputs (Name, Phone, Age, Pregnancy Status) via getline(), it checks if the input is empty. If true, the system retains the existing value, minimizing redundant typing for the receptionist.

- Phase 3: Advanced Data Validation and Conflict Resolution

Every new input undergoes validation before being accepted. Ages are captured as strings and parsed using stoi(), eliminating terminal crashes caused by invalid character inputs. Also when updating the phone number, the system calls searchPatientByPhone() to check for duplicates. If a duplicate phone number is found, it verifies whether the conflicting ID matches the current patient's ID. If they match, it simply means the patient is retaining their own number, and the system allows it to pass. Others, the system blocks the update to prevent entity merging.

- Phase 4: Confirmation and Priority Recalculation

Before committing any changes, the system presents a comparative summary and requires an explicit y/n confirmation - acting as a fail-safe against accidental modifications. Once confirmed, the temporary variables overwrite the node's data. Finally, the system automatically triggers the determinePriorityLevel() function to recalculate the patient's priority level, ensuring the Priority Queue algorithm will function correctly in subsequent operations.

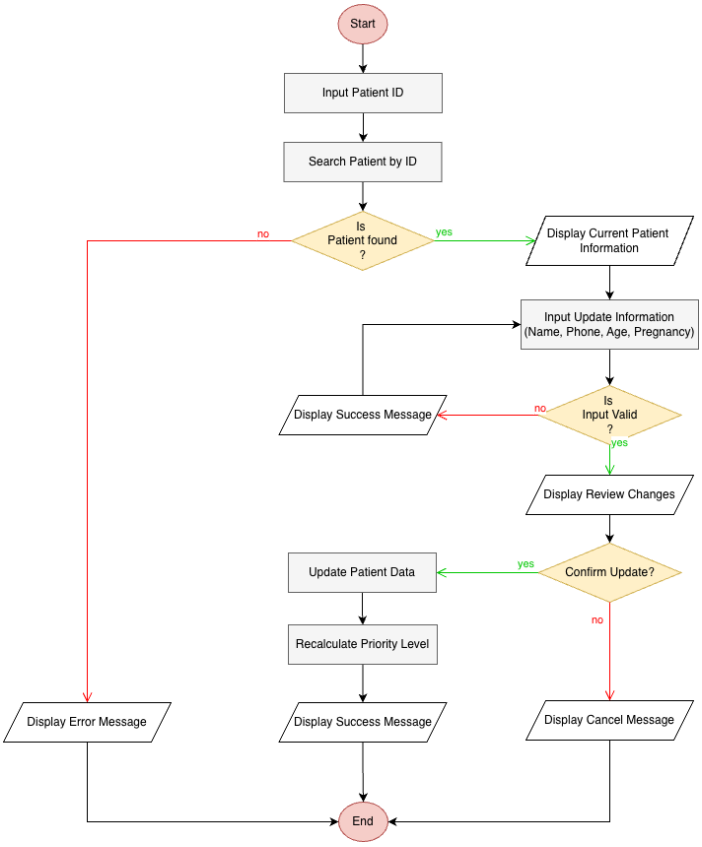

Figure 4: Update Patient Information Flowchart

1.4. Delete Patient

The deletePatient() function removes a patient record based on the patient ID. Since patients are stored in a singly linked list, deletion is performed by locating the target node, updating the pointer links, and releasing the node from memory.

The function first checks whether the list is empty. If *head == nullptr, it stops and displays an information message. If the target patient is at the head node, the head pointer is updated to the next node before deleting the original head. This is why the function uses Node** head, as the actual head pointer may change during deletion. For patients located after the head node, the function traverses the list using two pointers: temp to find the target node and prev to track the previous node. When the target node is found, the list is reconnected as follows:

```
prev->next = temp->next;
temp->data.appointments.clear();
delete temp;
```

This removes the node without shifting elements, unlike array-based deletion. The appointment vector is explicitly cleared before deletion, while its resources are also managed automatically when the containing Patient object is destroyed by delete temp.

The best-case time complexity is O(1) when deleting the head node. The worst-case time complexity is O(n) when the function must traverse the list to find the patient or confirm that the patient does not exist. The space complexity is O(1) because only pointer variables are used.

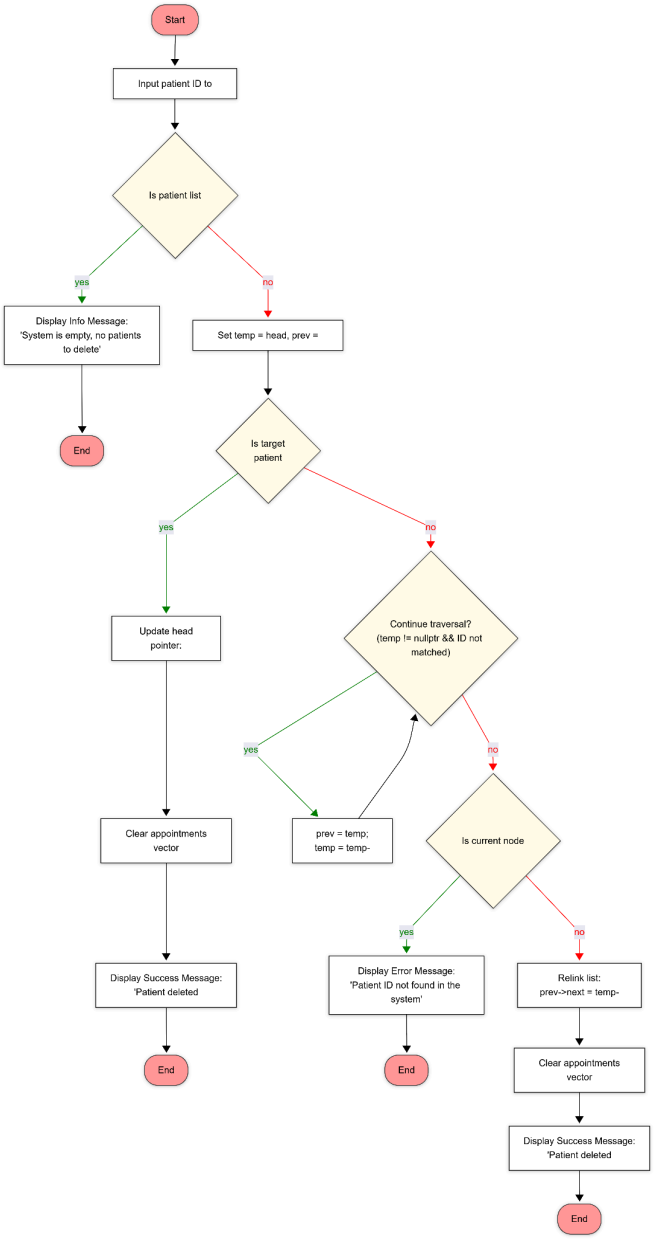

Figure 5: Delete Patient Flowchart

2. Appointment Management Module

2.1. Register Appointment

The register appointment workflow is the core operation of the appointment management module. Its purpose is to create a new appointment for an existing patient while enforcing scheduling rules, preventing double-booking, and ensuring that the selected doctor belongs to the correct department. The execution flow is structured into several phases to maintain both data integrity and booking consistency.

- Phase 1: Patient identification by phone number

The process begins by prompting the user to enter the patient's phone number. In the current system design, the appointment is not created by manually re-entering full patient details. Instead, the system uses the phone number as the primary lookup key because it is more practical for reception staff and less error-prone than searching by name. The input is captured using *getline()* and sanitized with *triminput()* to remove unwanted spaces. The system then calls *searchpatientbyphone(head, phonenumber)* to locate the corresponding patient node in the singly linked list. If no matching patient is found, the workflow terminates immediately and informs the user that the patient must be registered first before booking an appointment.

- Phase 2: Appointment data input and validation

Once the patient is successfully identified, the system prompts the user to enter the appointment information, including the appointment date, time, department, and doctor's name. Each input passes through a validation gate to ensure correctness before any appointment object is created. The date is validated by isvaliddate() to confirm that it follows the *yyyy-mm-dd* format and represents a real calendar date. The time is validated by *isvalidtime()* to ensure that it follows the *hh: mm* 24-hour format. The department and doctor name are also checked to ensure that they are not empty. This step prevents invalid appointments from entering the system and reduces the chance of runtime errors or corrupted records.

- Phase 3: Doctor-department verification and conflict detection

After the basic inputs are validated, the system performs a more advanced consistency check using the *validated doctor and department (doctorname, department)*. This function confirms that the doctor exists in the predefined hospital doctor list and belongs to the selected department. This prevents users from assigning an appointment to a doctor who does not work in that department.

Next, the system checks for booking conflicts in two directions. First, *isduplicateappointment(head, patientid, date, timeslot)* ensures that the same patient is not scheduled for two appointments at the same date and time. Second, *isdoctoravailable(head, doctorname, date, timeslot)* scans the entire linked list and all appointment vectors to verify whether the selected doctor is already booked at that exact time. This conflict function is essential because the system must prevent both patient double-booking and doctor double-booking.

- Phase 4: Automatic doctor substitution when the selected doctor is busy

A notable improvement in the current design is the automatic backup-doctor mechanism. If the chosen doctor is already occupied at the selected time, the system does not immediately reject the booking. Instead, it scans for another available doctor within the same department. This scan is done by iterating through the hospital doctor list and checking whether another doctor in the same department is free on the same date and time. If an alternative doctor is found, the system asks the user whether they want to switch to that doctor.

This design improves usability because the receptionist does not need to restart the booking process from the beginning. If the user accepts the alternative doctor, the system redirects the appointment to the new doctor. If the user declines, the workflow returns to the input stage so that the receptionist can choose a different schedule manually.

- Phase 5: Appointment creation and automatic field assignment

Once all validation and conflict checks have passed, the system creates a new *appointment* object. Several fields are assigned automatically rather than being manually entered by the user. The system generates *appointmentid* using the patient ID and the current timestamp to ensure uniqueness. The *status* is initialized as *"waiting"* because the appointment is newly created and not yet completed. The *queuenumber* is initially set to 0, since the exact queue position will later be determined by the sorting module. The *registrationtimestamp* is recorded using system time so that the sorting algorithm can later apply the first-come, first-served rule within the same priority level. Finally, the appointment is pushed into the patient's *vector<appointment>* using *addappointment(p, a).* At this point, the booking is officially stored in memory and becomes part of the patient's appointment history.

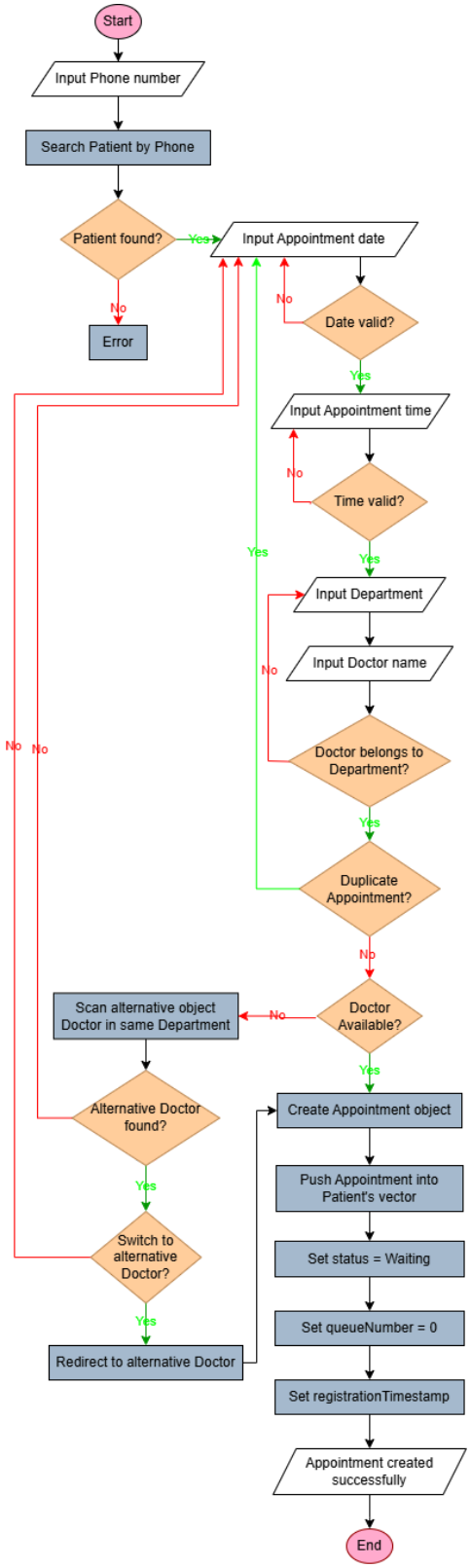

Figure 6: Register Appointment

The computational complexity of the appointment registration workflow depends primarily on the conflict detection phase. The function *searchPatientByPhone()* performs a linear search on the singly linked list with a worst-case time complexity of O(n)**,** where n represents the total number of patients.

Additionally, the *isDoctorAvailable()* function traverses the entire linked list and scans every appointment vector to detect scheduling conflicts, resulting in a worst-case complexity of O(n × m)**,** where m represents the average number of appointments per patient. The insertion into *std::vector* using *push_back()* operates with amortized O(1) time complexity. Therefore, the overall appointment registration workflow is dominated by the conflict scanning operations.

In terms of memory usage, the workflow requires only a small number of temporary variables and one dynamically appended appointment object. Thus, the auxiliary space complexity is O(1), excluding the permanent storage allocated for the newly created appointment inside the vector.

2.2. Doctor Schedule

The doctor schedule function is designed to display all appointments assigned to a specific doctor on a specific date. This feature is useful for reception staff and for managing the daily working queue because it allows the system to filter the waiting list based on doctor and date, instead of showing the entire patient database.

- Phase 1: Input processing and schedule target definition

The workflow begins by prompting the user to enter a doctor's name and a target date. Both inputs are sanitized using *triminput()* to remove extra spaces. The purpose of this step is to define the exact schedule that needs to be displayed. Instead of traversing the system blindly, the function now has a precise search target: one doctor on one date.

- Phase 2: Linked list traversal and appointment filtering

The system then traverses the singly linked list from the head node to the end. For each patient node, it loops through the patient's *vector<appointment>*. The function compares the doctor's name and the appointment date against the user's search criteria. Since the schedule is stored in a dynamic vector rather than a fixed array, each patient may have zero, one, or many appointments. This makes Vector a suitable choice for appointment history because it supports flexible growth without predefining a capacity limit.

Only appointments that match both the doctor's name and the selected date are included in the output. This filtering behaviour makes the function efficient for displaying only the relevant schedule rather than all patient data.

- Phase 3: Waiting list and queue information display

For every matching appointment, the system displays the appointment time, patient name, patient ID, queue number, and status. In the current design, *queuenumber* is important because it represents the patient's position in the doctor's daily waiting queue. If the sorting module has already run, queue numbers are assigned according to priority level and registration time. This allows the schedule display to reflect the actual order in which patients should be served.

In addition, the system also counts the total number of patients waiting for that doctor on that date. This gives the receptionist an immediate overview of the workload for a particular doctor and helps with daily queue management.

- Phase 4: Result evaluation and user feedback

If no matching appointments are found, the system displays a message stating that the doctor has no appointments on that date. Otherwise, the schedule is shown clearly with all relevant appointment details. This function acts as a read-only reporting tool, so it does not modify any data in the linked list or appointment vectors. Its role is purely to support queue visibility, staff coordination, and daily schedule management.

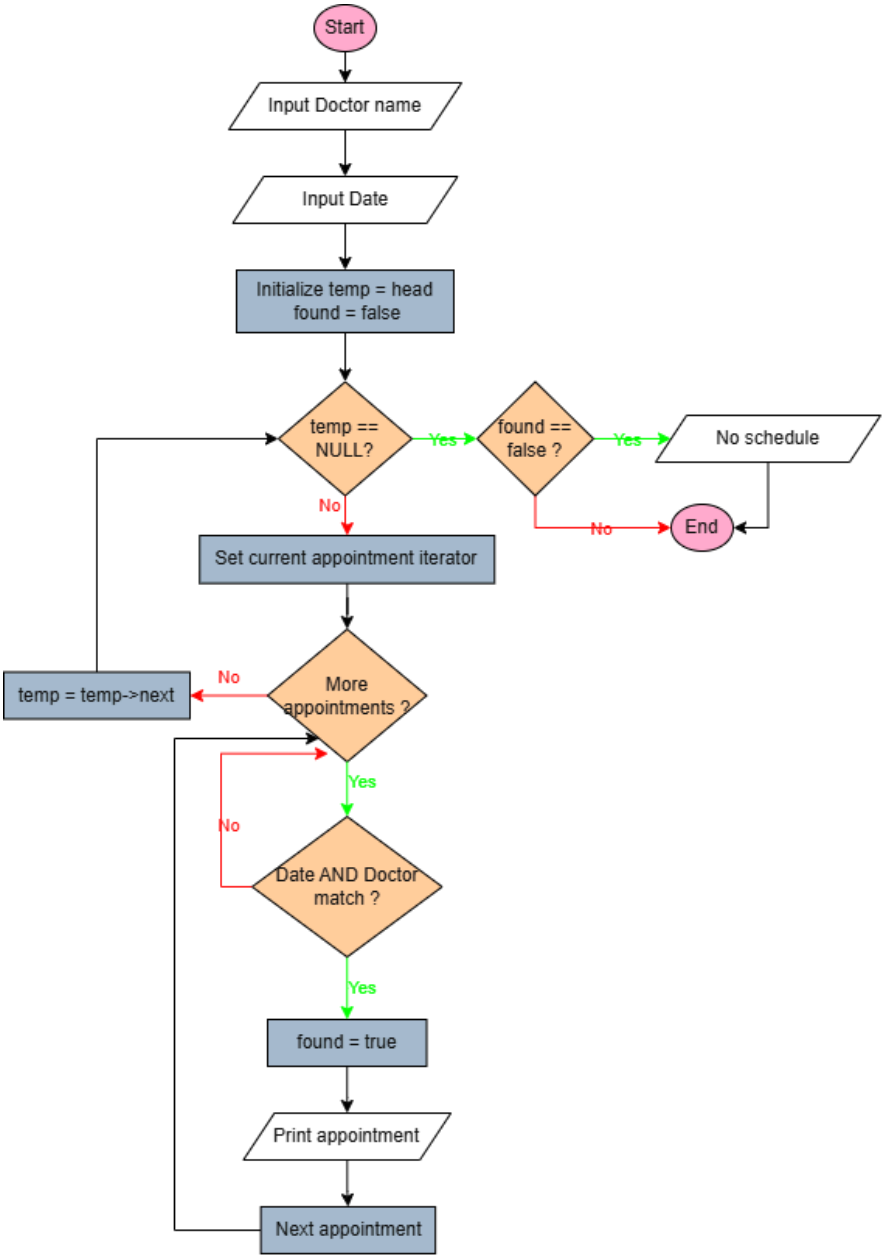

Figure 7: Doctor schedule

The doctor schedule workflow relies on a complete traversal of the singly linked list and the nested appointment vectors. In the worst-case scenario, the function iterates through all patients and all appointments to locate matching schedules. Therefore, the overall time complexity is O(n × m), where n is the number of patients and m is the average number of appointments stored per patient.

Because the workflow only performs filtering and display operations without allocating additional dynamic data structures, the auxiliary space complexity remains O(1).

3. Queue Processing Module

3.1. Sort Waiting List

Functions: sortWaitingList(Node** head), mergeSort(), splitList(), sortedMerge(), shouldComeBefore(), getEarliestActiveTimestamp()

Algorithm:

- Uses a custom implementation of Merge Sort on the Singly Linked List of patients.
- The `mergeSort` function recursively splits the list into two halves using a slow and fast pointer approach (“splitList”), sorts the sub-lists, and merges them back together (“sortedMerge”).
- Time Complexity: O(n log n) – highly efficient for linked lists, as it provides stable, fast sorting and does not require extra contiguous memory allocation (unlike array sorting).

Sorting logic is governed by a comparison function “shouldComeBefore(Patient a, Patient b)”. It evaluates patients based on a two-tier rule system:

- Primary Rule (Priority Level): The system first compares the `priorityLevel` attribute. A lower numerical value represents a higher priority (e.g., Priority 1 is served before Priority 2). Priority 1 typically includes vulnerable groups such as children under 5, the elderly over 70, and pregnant women.
- Secondary Rule (First-Come, First-Served - FIFO): If two patients share the exact same priority level, a tie-breaker is required. The system calls “getEarliestActiveTimestamp()”, which scans the patient's dynamic vector of appointments, checking for any active statuses ("waiting", "scheduled", or "pending"). It finds the earliest “registrationTimestamp” (recorded in UNIX time) among those active appointments. The patient with the earlier timestamp is placed first.

Edge Case Handling: If a patient has no active appointments, their earliest timestamp is safely defaulted to the maximum possible value (“LONG_MAX"). This pushes them entirely to the back of the waiting list so they don't block active patients.

Post-Sorting: Once “mergeSort” finishes reordering the list, the system iterates through the newly sorted list. It scans each patient's appointment vector and assigns a sequential “queueNumber” (starting from 1) only to appointments that explicitly have a "waiting" status.

3.2. Queue Position

Functions: calculateQueueNumber(), calculateWaitingCount()

The queue positioning system provides visibility into the sorted waiting list without modifying the list structure. It relies on the pre-calculated “queueNumber” values assigned during the sorting phase.

calculateQueueNumber(Node head, string patientID):*

This function sequentially traverses the singly linked list to locate the specific patient by their ID. Once found, it iterates through the patient's dynamic “vector<Appointment>”. It specifically looks for an appointment with a "waiting" status (using case-insensitive string matching to ensure robust comparison).

- If a waiting appointment is found, it retrieves its "queue number". A value greater than 0 confirms the sorting algorithm has successfully assigned a position, which is then returned.
- If the patient is found but has no active waiting appointments, or if the patient ID does not exist in the system at all, the function returns `-1` to indicate an invalid or non-applicable queue status.

calculateWaitingCount(Node head, int queueNumber):*

A mathematical helper function designed to improve the user interface. It takes the actual queue position returned by “calculateQueueNumber” and calculates how many patients are ahead.

- If the provided queue number is valid (greater than 0), it simply returns “queueNumber - 1". For example, if a patient is 5th in the queue, 4 people are waiting ahead of them.
- If the input is 0 or negative, it returns “-1” as an error indicator.

Interation

- Menu Option 7: Explicitly sorts the list and displays the new order.
- Menu Option 8: Triggers an automatic sort behind the scenes to ensure accuracy, then fetches the queue position for a given patient.

4. Data Persistence Module

FILE I/O STRATEGY

Data is stored in a flat text file named “patients_data.txt”. Because patients can have a variable number of appointments (stored in a dynamically sized std::vector), a multi-line, pipe-delimited ('|') string serialization format is used rather than a rigid matrix.

SAVE TO FILE

Function: saveToFile(Node* head)

- Iterates through the singly linked list of patients.
- Writes core patient info (ID, name, age, phone, pregnancy status, and priority level on a single line.
- The critical final field on the patient line is “appointments.size() ".
- For each appointment, it writes a new line directly below the patient line with the appointment's details (ID, date, time slot, department, doctor name, queue number, status, and registration timestamp).

LOAD FROM FILE

Function: loadFromFile(Node** head)

- Read “patients_data.txt” line by line using `std::stringstream`.
- Parses the main patient details first.
- Uses the previously saved “appointments.size()” count to loop exactly that many times to read the subsequent appointment lines.
- Reconstructs the “std::vector<Appointment>” and dynamically adds the patient to the linked list using "addPatient()".
- Safe handling: If no file is found, it cleanly starts fresh without crashing.

INTEGRATION

- Startup: “loadFromFile(&head)” is called immediately when the application starts, right before displaying the main menu.
- Exit: “saveToFile(head)” is called when the user selects Option 0 (Exit) to ensure the latest state is written to disk before memory is freed.

IV. Results

The test scenario specification for the Patient Waiting List Management System. Testing follows a conflict-driven methodology: each test case is anchored to a specific business-rule conflict ID

identified during code review. Every test case was executed against the compiled binary and the actual terminal output was captured to determine the real PASS or FAIL result.

PASS = actual output matches the expected behaviour defined by the business rule.

FAIL* = a known conflict confirmed by execution; documented as a sprint limitation with rationale.

TC	Test Case Name	Verdict
TC-01	Priority boundary - age = 5	PASS
TC-02	Priority boundary - age = 70	PASS
TC-03	Priority auto-assign - age = 4 & age = 72	PASS
TC-04	Priority auto-assign - pregnant	PASS
TC-05	updatePatientrecalculates priority; queue not auto-re-sorted	PASS
TC-06	Doctor name — double-space bypasses conflict detection	PASS
TC-07	Doctor name — case-insensitive match works	PASS
TC-08	Register valid appointment	PASS
TC-09	Duplicate slot — same patient, same date/time	PASS
TC-10	Duplicate slot — same patient, same time, different doctor	PASS
TC-11	Doctor conflict — same doctor, same date/time, different patient	PASS
TC-12	Validation — invalid date format (DD/MM/YYYY)	PASS
TC-13	Validation — age = 0 rejected	PASS
TC-14	Validation — duplicate phone not checked	PASS
TC-15	Sort — all priority-1 before priority-2	PASS
TC-16	Sort — FIFO within same priority	PASS
TC-17	Sort — patient with no appointments appears in sorted list	PASS
TC-18	Queue number — patient with 2 Waiting appts gets 2 queue numbers	PASS

TC-19	Display doctor schedule — case-insensitive	PASS
TC-20	Save to file — pipe-delimited format correct	PASS
TC-21	Load from file — state fully restored after restart	PASS
TC-22	Delete patient — valid & invalid ID	PASS

Challenges Encountered During Testing

The program uses a blocking while(true) menu loop. Test input had to be crafted as a complete piped sequence (printf '...' | ./app) including an exit route for cases that loop on validation errors. Missing an exit caused the test process to hang.

State isolation between test cases because saveToFile() runs on every exit (option 0), each test has to restore patients_data.txt from a backup before running. This was done with 'cp patients_data_backup.txt patients_data.txt' before each test.

V. Conclusion

The development of the Patient Waiting List Management System successfully digitized a complex clinical workflow using foundational computer science principles. By implementing a Singly Linked List coupled with a hybrid vector structure, the project efficiently handled dynamic memory allocation for unpredictable patient volumes. The integration of robust data validation, automated ID generation, and a smart triage scheduling algorithm resolved real-world issues of double-booking and administrative overhead.

While the current system operates efficiently via a Command Line Interface (CLI) and text-file persistence, future enhancements could involve migrating the backend to a Relational Database Management System (RDBMS) like MySQL and developing a Graphical User Interface (GUI) to further improve user experience.

REFERENCES

[1] HIPAA Journal, "Healthcare data breach statistics," 2026. [Online]. Available: https://www.hipaajournal.com/healthcare-data-breach-statistics/

[2] L. Zhang and R. Kumar, "Adaptive double-booking strategy for outpatient scheduling using individualized no-show prediction," *arXiv*, 2026. [Online]. Available: https://arxiv.org/pdf/2603.07270.pdf

[3] B. Davis, "Real-time priority scoring system must be used for prioritisation on waiting lists," *BMJ*, vol. 318, no. 7199, p. 1699, 1999.

[4] World Health Organization, "Recommendations on digital interventions for health system strengthening," 2019. [Online]. Available: https://www.ncbi.nlm.nih.gov/books/NBK541902/

[5] Vivekanandha College of Engineering, "Introduction to data structures assignment I: Case study – Patient medical records management," *Scribd*, 2025. [Online]. Available: https://www.scribd.com/document/897752619/U23IT409-Introduction-to-DS-ASSIGNMENT-1-Docx

[6] HCEM, "Giáo trình cấu trúc dữ liệu và giải thuật," 2025. [Online]. Available: https://hcem.edu.vn/wp-content/uploads/2025/10/MH10_Cau-truc-DL-va-Giai-thuat.pdf

[7] L. Zhang and R. Kumar, "Adaptive double-booking strategy for outpatient scheduling," *arXiv*, 2026.